

Patient Message Generation – Full Documentation

Objective

Design and implement a system that generates empathetic, accurate, and personalized post-session messages from a Physical Therapist (PT) to each patient based on structured session data. The messages should be warm, short (2–4 sentences), human, and ready for PT review — where they can be accepted, edited, or rejected with tagged reasons like "Tone", "Factuality", or "Generic".

System Structure

1. `get_system_prompt()`

Purpose

Provides high-level behavioral instructions to the assistant (LLM) about the role it is playing and the expectations for tone, content, and format.

Prompt Goals

- Simulate a real, warm, encouraging PT.
- Generate messages that feel human, motivational, and empathetic.
- Avoid all references to AI, automation, or system origins.
- Conform to clinic-style brevity: 2–4 sentences.
- Adapt tone based on performance or struggle indicators (e.g., pain, fatigue, dropout).

Design Assumptions

- The assistant is given structured, relevant patient session data.
- The assistant should always sound like a trusted clinician.
- The model does not see message history or long-term patient profile.

Reasoning

Most PTs don't want to edit long or robotic messages. By explicitly instructing the model to use warmth, brevity, and encouragement — and never mention automation — we align output with PT expectations and reduce rejection rates.

2. `get_user_prompt(features: dict)`

Purpose

Builds a detailed, human-readable session summary that the assistant can use to craft a message specific to each patient.

Prompt Behavior Instructions (given to LLM)

- Address the patient by first name only.
 - Begin with an energetic and friendly opener (e.g., “Amazing work, Haley!” not “Dear Haley”).
 - Vary the opening line to avoid repetitive phrasing across sessions.
 - Mention performance, effort, or progress — not just raw stats.
 - If any pain, fatigue, or dropout occurred → offer supportive encouragement.
 - Close with a soft sign-off like “Your PT, Jamie”.
 - Do **not** mention AI, automation, or data pipelines.
-

Prompt Construction: Design Considerations

Input Fields

The **features** dictionary contains session-level patient data such as:

- **Patient metadata:** `patient_name`, `pt_name`, `therapy_name`, `session_number`
- **Performance metrics:** `pain`, `fatigue`, `perc_correct_repeats`, `training_time`, `number_exercises`, etc.
- **Session metadata:** `session_group`, `session_is_nok`
- **Quality indicators:** `quality_reason_movement_detection`, `quality_reason_exercises`, `quality_reason_easy_of_use`, etc.
- **Dropout reasons:** `leave_exercise_pain`, `leave_exercise_tired`, `leave_exercise_difficulty`, etc.

Prompt Structure

Organized into four readable sections:

1. **Patient Info** – Personalization and clinical context.
2. **Session Performance** – Drives tone (celebratory vs. supportive).
3. **Quality Feedback** – Suggests usability issues or frustration.
4. **Dropout Reasons** – Flags skipped segments that may need empathy or re-engagement.

Key Design Features

- Only the **first name** is used via `split()[0]` to preserve a warm, informal tone.
 - `.get()` is used for all field access to guard against missing or **None** values.
 - Message summaries are presented as **natural language notes**, not key-value dumps.
 - Prompts are **fully instructive**, guiding the assistant’s tone, format, and framing choices.
 - The assistant is encouraged to **vary its opening phrasing** to reduce repetition.
-

Future Improvements

- Fine tuning.
- Allow message length/tone to be customized per therapist or patient preference.
- Introduce templates or controlled generation when tone/style needs to be locked for compliance.
- Build a UI preview mode where PTs can simulate how the message would change with different inputs.

- Integrate conversation memory (while preserving privacy) for continuity across multiple sessions.
 - Include multilingual support for non-English patients while retaining tone and cultural nuances.
 - Continuous Evaluation: Include weekly PT feedback metrics (e.g., rejection rate, edit distance) to guide iterative improvements.
 - Batch Processing: Extend the message generation pipeline to support bulk inference across all patient sessions daily, rather than single requests.
 - Caching: Use hash-based caching of generated messages to prevent regeneration for unchanged sessions.
 - Model Hosting: Host the model behind an API with autoscaling (e.g., using FastAPI + Ray or serverless infrastructure) to handle peak demand from clinics.
-